Perplexity Holds, Programs Break: Executable Correctness as a Blind Spot of KV-Cache Compression

Prajjwal Chittori

Independent

June 23, 2026

Abstract

KV-cache compression methods are evaluated almost exclusively with metrics that score token or string overlap, retrieval hits, or an extracted final answer. None of the widely used long-context suites check whether generated output actually *runs*, or whether a generated tool call is *well-formed*. We argue this is a systematic blind spot: lexical and retrieval metrics can stay flat while a compressed cache quietly breaks structured generation, because a single dropped or garbled token fails a unit test or schema check but barely moves a substring score. We introduce **kv-exec-bench**, a small open benchmark that measures the two missing quantities, code unit-test pass@1 and tool-call JSON-Schema validity, under KV compression. The benchmark is built on top of NVIDIA's `kvpress` as a dependency, so any press works without modification. In a deliberately small, CPU-only study on a 0.5B model we observe the predicted divergence on tool calling: for the best-behaved presses, the rate of *parseable* tool calls stays at 100% across compression ratios while the rate of *schema-valid* calls falls by half or more. A parse-only or string-only metric would score this regime as nearly lossless. (The code task floors at this model size and is released for use at scale.) We release the benchmark and all code under Apache-2.0 and frame this as a measurement-gap report plus a reusable benchmark, not a large-scale empirical study.

1 Introduction

KV-cache compression is now a standard lever for long-context LLM serving: by evicting, merging, or approximating key/value entries, a server fits longer contexts and larger batches into fixed memory. A method is judged by how little it costs in quality at a given compression ratio. The implicit contract is "compression is approximately lossless" up to some ratio.

That contract is measured with a narrow set of instruments. The long-context suites used to validate KV compression, RULER [5], LongBench [2], ∞Bench [12], needle-in-a-haystack probes [6], and various math sets, all reduce a generation to a token/string overlap, a retrieval hit, or an extracted answer span. These instruments share a failure mode: they are insensitive to the kind of damage that destroys *structured* output. A function body that is one token wrong still overlaps almost entirely with the reference, yet fails every unit test. A tool call with a malformed `arguments` object is unparseable, yet may still contain the right tool name as a substring.

This paper makes three contributions:

- **C1 (the gap).** We show that the metrics used to validate KV-cache compression do not measure executable or structural correctness, and argue why this is precisely the failure mode compression is likely to induce.

- **C2 (the benchmark).** We release `kv-exec-bench`, which measures code unit-test pass@1 and tool-call JSON-Schema validity under any `kvpress` [8] press, with eval-grade execution isolation for the code task.
- **C3 (a demonstration).** In a small, CPU-only study we show the predicted divergence: string-overlap anchors stay high as the compression ratio rises while executable and structured metrics collapse.

We are explicit about scale. All experiments here run on a single laptop-class CPU (Section 4); the contribution is the measurement gap and a reusable benchmark, with a small demonstration that the gap is real. Scaling the same harness to larger models and GPUs is straightforward future work.

2 Related Work

KV-cache compression. StreamingLLM [10] keeps attention sinks plus a recent window; H2O [13] evicts by accumulated attention ("heavy hitters"); SnapKV [7] selects important prefix positions using an observation window; PyramidKV [3] allocates budget non-uniformly across layers. The `kvpress` [8] library provides a unified pipeline that implements these and others as interchangeable presses; we build on it directly so the benchmark is press-agnostic.

How compression is evaluated. The suites above [5, 2, 12, 6] dominate KV-compression evaluation. Every metric they expose is a string-overlap, retrieval, or extracted-answer score. To our knowledge none reports whether generated programs execute correctly or whether generated tool calls validate against a schema. That is the gap we target.

Code and tool-call evaluation. Functional-correctness evaluation is standard outside the compression literature: HumanEval [4] and MBPP [1] score unit-test pass rate, and the Berkeley Function-Calling Leaderboard [11] scores tool-use correctness. We import these notions of correctness into the KV-compression setting.

3 Benchmark Design

`kv-exec-bench` consists of two tasks that share a column contract (`context, question, answer_prefix, answer, task, max_new_tokens`) and a scorer per task.

tool_call. A shared catalog of tool definitions (name, description, JSON Schema) is the context. Because the catalog is shared across requests, a prefill press compresses it once and answers every request from the compressed cache, the regime where a dropped or garbled tool definition turns into an invalid call. The model is asked to emit a single JSON object naming one tool and its arguments. We extract the first balanced JSON object from the output and score four nested metrics, lenient to strict: `json_valid` (a JSON object was extracted), `name_match` (correct tool), `schema_valid` (arguments validate against the tool's JSON Schema), and `args_exact` (arguments exactly equal the gold values). As a string anchor we also record `name_substring`: whether the gold tool name appears anywhere in the raw output, regardless of JSON validity. No code is executed.

code_exec. The model completes a HumanEval [4] function signature and docstring. We assemble `prompt + completion + reference tests` and run it, scoring unit-test `pass@1`. As a string anchor we record `edit_sim`, the difflib ratio between the completion and the canonical solution. Execution is opt-in (`KV_EXEC_BENCH_ALLOW_CODE_EXECUTION=1`) and isolated: a fresh temporary working directory, a stripped environment, a wall-clock timeout, and, on POSIX, CPU-time and address-space rlimits. This is eval-grade isolation for trusted-but-buggy model output, not an adversarial sandbox.

The contrast. For each task we pair a string anchor (`name_substring`, `edit_sim`) with a structural/executable metric (`schema_valid`, `pass@1`). The central empirical question is whether, as the compression ratio rises, the anchor stays high while the structural metric falls. If so, a string-only evaluation would report compression as near-lossless exactly where functional correctness has collapsed.

4 Experimental Setup

Hardware (disclosed in full). All experiments were run on a single CPU-only machine: an AMD Ryzen 5 5500U (6 cores / 12 threads, AVX2, no AVX-512), 14 GiB RAM, Ubuntu 22.04 (Linux 6.8), a PyTorch CPU build, and no GPU. This bounds the study to small models and a small problem count; we state this as the primary limitation rather than hide it.

Models. We evaluate `Qwen2.5-0.5B-Instruct` [9], the largest model that fits comfortably in this machine's memory for full-precision CPU inference. Larger models from the same family (1.5B and up) did not fit alongside the working set, which is itself a consequence of the single-box constraint.

Presses and ratios. We run an uncompressed baseline (`none`) plus five `kvpress` presses, `Random` (a floor), `StreamingLLM` [10], `SnapKV` [7], `Knorm`, and `ExpectedAttention`, each at compression ratios 0.25, 0.5, and 0.75.

Data. For `tool_call` we use the benchmark's built-in inline catalog of twelve tools with nine concrete requests (network-free); integrating the Berkeley Function-Calling Leaderboard [11] is left as the headline follow-up. For `code_exec` we use a slice of ten HumanEval problems.

5 Results

5.1 tool_call: well-formedness survives, correctness does not

Figure 1 shows the central result. Averaged over all five presses, `json_valid` retains 0.58 of its uncompressed value at compression ratio 0.75, while `schema_valid` retains only 0.20. The model keeps producing things that parse as tool calls; those calls are increasingly wrong.

The per-press breakdown (Table 1) is sharper. For the two best-behaved presses, `Knorm` and `ExpectedAttention`, `json_valid` never drops below 1.0 at any ratio: every output is a parseable tool call. Yet over the same ratios `schema_valid` falls from 0.78 (uncompressed) to 0.33, and the strictest metric `args_exact` (exact argument match, not shown) falls to 0 for most presses by ratio 0.5. A parse-rate metric would score these presses at or near 100% success precisely where a third to two-thirds of the tool calls have become semantically invalid. `SnapKV` behaves similarly (`json_valid` $\approx$ 1.0, `schema_valid` 0.44 $\rightarrow$ 0.11). `StreamingLLM` collapses entirely on this model

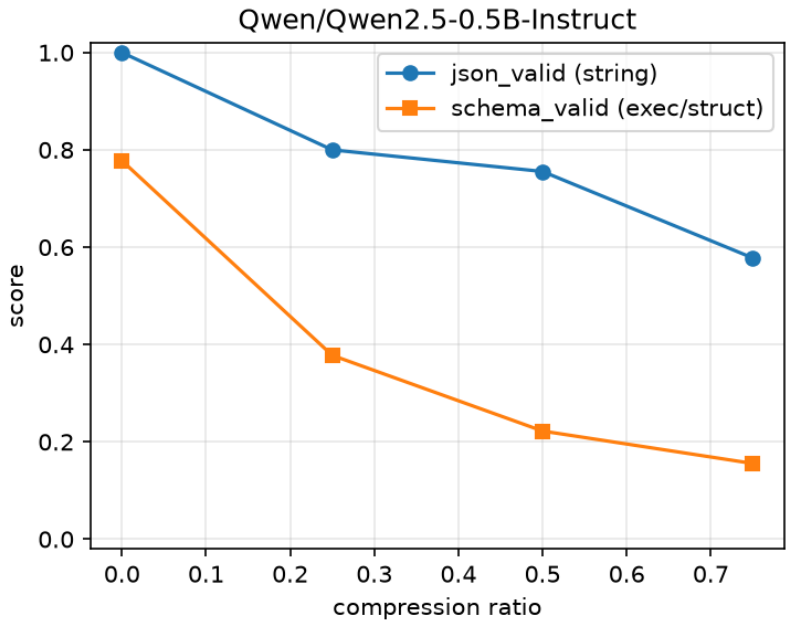

Figure 1: `tool_call` on Qwen2.5-0.5B-Instruct, averaged over the five presses. A shallow well-formedness check (`json_valid`: did the model emit a parseable tool call?) stays high as the compression ratio rises, while `schema_valid` (are the call's arguments actually valid for the tool?) collapses. A string-only or parse-only evaluation would report this regime as nearly lossless.

(`json_valid` = 0 from ratio 0.25), and `Random` degrades quickly as expected of a floor; we report both rather than hide the outliers.

5.2 code_exec at this scale

On the ten-problem HumanEval slice, Qwen2.5-0.5B-Instruct scores `pass@1`= 0 even uncompressed, so compression cannot make it worse and the task yields no contrast at this model size (the string anchor `edit_sim` is likewise near zero, $\approx$ 0.08, because the small instruct model emits prose-wrapped rather than canonical completions). We report this honestly: the `code_exec` harness runs, executes, and scores correctly, but a 0.5B model on a CPU is too weak to produce a meaningful executable-correctness signal. Demonstrating the `code_exec` contrast requires a capable code model on a GPU, which we leave as the immediate follow-up. The benchmark is released with both tasks.

6 Discussion and Limitations

Scale. The headline limitation is scale: a single CPU box, small models, and a small problem count. We therefore do not claim quantitative degradation rates for production-scale models; we claim that the measurement gap is real and that the divergence appears even at this scale. The harness is press- and model-agnostic, so reproducing at scale on a GPU is a configuration change.

Prefill-only regime. We study prefill-time compression of a shared context. Decode-time compression and per-request contexts are out of scope here.

	json_valid (shallow)				schema_valid (correct)		
press	base	0.25	0.5	0.75	0.25	0.5	0.75
none	1.00	–	–	–	0.78 (base)		
Random	1.00	1.00	0.78	0.00	0.00	0.00	0.00
StreamingLLM	1.00	0.00	0.00	0.00	0.00	0.00	0.00
SnapKV	1.00	1.00	1.00	0.89	0.44	0.11	0.11
Knorm	1.00	1.00	1.00	1.00	0.89	0.67	0.33
ExpectedAttention	1.00	1.00	1.00	1.00	0.56	0.33	0.33

Table 1: Per-press `tool_call` on Qwen2.5-0.5B-Instruct. The uncompressed baseline is `json_valid`= 1.00, `schema_valid`= 0.78. For `Knorm` and `ExpectedAttention`, `json_valid` stays at 1.00 across all ratios while `schema_valid` falls by half or more: well-formedness is preserved, correctness is not.

Isolation. `code_exec` isolation is eval-grade, not adversarial; it should be run on throwaway or CI hardware.

Threshold framing. Where a structural metric holds at low ratios and falls at higher ratios, the useful reading is a degradation *onset*: the compression ratio at which functional correctness departs from what string metrics would suggest.

7 Conclusion

KV-cache compression is validated with instruments that cannot see the damage it is most likely to cause to structured generation. `kv-exec-bench` supplies the missing instruments, code pass@1 and tool-call schema validity, on top of `kvpress`. Even a small CPU-only study shows string anchors holding while executable correctness drops. We hope the benchmark makes executable correctness a routine axis when reporting KV-compression results.

Availability. Code and benchmark: `https://github.com/pjdurden/kv-exec-bench` (Apache-2.0).

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[2] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[3] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, and Wen Xiao. PyramidKV: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024.

[4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[5] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In *Conference on Language Modeling (COLM)*, 2024.

[6] Greg Kamradt. Needle in a haystack - pressure testing llms. `https://github.com/gkamradt/LLMTest_NeedleInAHaystack`, 2023.

[7] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: Llm knows what you are looking for before generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[8] NVIDIA. kvpress: Llm kv cache compression made easy. `https://github.com/NVIDIA/kvpress`, 2024.

[9] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[10] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.

[11] Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Tianjun Zhang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Berkeley function calling leaderboard. `https://gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html`, 2024.

[12] Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Khai Hao, Xu Han, Zhen Leng Thai, Shuo Wang, Zhiyuan Liu, and Maosong Sun. ∞bench: Extending long context evaluation beyond 100k tokens. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[13] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.